

Nombres	Oscar Eduardo Arias Peralta	
Fecha de evaluación	10/22/25	
Fecha de ingreso	7/1/25	
Stack tecnológico	Javascript, Angular, PHP, Laravel, CMS (Wordpress, Magento, Joomla, Liferay). Experiencia frontend de 3 años	
Proyección de crecimiento	Corto plazo: pulir temas técnicos y mejorar en desarrollo mobile. Mediano plazo: hacer maestría en IA Largo plazo: liderazgo de proyectos	
Observaciones	Oscar demuestra deseos de desafiarse constantemente. Faltan conceptos y metodología.	
Nivel		
Subnivel		

Superpoderes		Recursos				Observaciones
Protocolos	Comprende la naturaleza asíncrona de hacer solicitudes y recibir recursos sobre un canal HTTP, mediante Fetch y/o XMLHttpRequest, intercambiando información con recursos remotos mediante el formato JSON.	Link				
Diseño	Es capaz de maquetar documentos estáticos a partir del modelo de cajas .	Link	Link			
Layout	Entiende el DOM y es capaz de manipular la estructura, estilo y contenido de sus nodos.	Link				
Lenguaje	Conoce los tipos de dato primitivos y objetos, y sus diferencias. Conoce las estructuras de control fundamentales en su código.	Link	Link	Link	Link	
Semántica	Entiende los métodos o verbos HTTP y la organización de los códigos de respuesta.	Link	Link	Link	Link	
Arquitectura	Entiende la programación orientada a eventos de JS y es capaz de resolver necesidades básicas desde la interacción del usuario. Conoce al menos dos formas de capturar los eventos del usuario.	Link	Link	Link		

Lenguaje	Es capaz de usar estructuras de datos fundamentales (arreglos, pilas, colas). Comprende el concepto del ámbito, contexto (scope) de las variables y es capaz de aplicarlo conscientemente en sus estructuras de control y funciones. Entiende el concepto y las implicaciones del Hoisting.	Link	Link			
Diseño	Es capaz de maquetar documentos a través del diseño adaptativo, de acuerdo a diversos dispositivos y navegadores Web. Conoce Mobile First.	Link				
Layout	Es capaz de reconocer cuándo aplicar la técnica Flexbox (contenedores) y cuándo la de CSS-Grid (documentos).	Link	Link			Brecha: profundizar conceptos
Principios	Sabe cómo nombrar variables, métodos y clases según los estándares de Código Limpio , y es capaz de comentar solo lo necesario. Codifica en inglés. Hace uso del tipado estático de Typescript .	Link	Link	Link		Brechas: Código Limpio. Todavía usa any.
Paradigmas de programación	Comprende la POO y puede representar modelos básicos mediante la abstracción de casos específicos de negocio. Conoce los principios de polimorfismo, herencia, encapsulamiento.	Link				
Control de versiones	Tiene el hábito de versionar su código fuente; clona repositorios, crea y se mueve entre ramas, realiza envíos a repositorios remotos y crea solicitudes de integración de código.	Link	Link			
Semántica	Estructura documentos mediante el uso correcto de la semántica HTML , aplicando técnicas para SEO cuando corresponda.	Link	Link			
Frameworks	Posee experiencia con un framework basado en componentes, que le permite hacer enrutamiento y manipular el DOM, junto con la integración de soporte HTTP, un empaquetador y una herramienta para la administración de dependencias.	Link	Link			Brechas: empaquetadores, enrutamiento
Codificación	Comprende la problemática de la mutabilidad , así como los diversos mecanismos de clonación superficial y profunda de objetos. StructuredClone() vs operador de propagación vs JSON.parse/stringify vs lodash.cloneDeep.	Link	Link			
Lenguaje	Aplica expresiones y operadores modernos que simplifican y hacen más legible el código fuente (propagación, desestructuración, Nullish coalescing (??), optional chaining (?), conversión de tipos, template strings, funciones flecha).	Link	Link			

Rendimiento	Implementa técnicas para la mejora de rendimiento Web como caché, selectores CSS anidados, virtual DOM, Signals, Resumability, SSR, lazy loading, compresión, minificación, uglificación, optimización de imágenes y desuscripciones. Inmutabilidad en la detección de cambios en frameworks. Desuscripción de observables.	Link	Link	Link		
Bases de datos	Ha utilizado manejadores de bases de datos NoSQL, SQL Lite o almacenamiento en el navegador. Diferencia entre sessionStorage y localStorage. IndexedDB.	Link	Link			
Frameworks	Conoce a profundidad y enseña un framework basado en componentes, que le permite hacer formularios reactivos, asegurar rutas (guards), crear funciones de propósito (directivas, pipes, hooks), dominar el ciclo de vida, utilizar preprocesadores CSS, usar interceptores, insertar componentes dinámicos, realizar cargas diferidas de módulos, precargar imágenes, usar shadow DOM y crear custom Hooks.	Link	Link	Link		Brechas: ciclos de vida, guards, interceptores, formularios reactivos y validaciones dinámicas Conoce: preprocesadores CSS, lazyloading shadowDOM
Pruebas	Escribe pruebas unitarias síncronas y asíncronas, mocks, y pruebas de interfaz de usuario E2E. Comprende las partes de la prueba (Patrón de las tres AAA).	Link	Link			
Codificación	Entiende los conceptos de sincronismo y asincronismo en JS, el hilo de ejecución y el EventLoop . Comprende los conceptos de Paralelismo (en múltiples hilos) y Concurrencia (en un hilo).	Link	Link			Brecha: eventloop, multihilos, paralelismo vs concurrencia
Arquitectura	Conoce el patrón de diseño Observador , comprende RxJS , operadores de transformación y combinación de observables. Identifica las diferencias entre promesas y observables .	Link	Link			
Arquitectura	Entiende y ha implementado Microfrontends . (Module Federation, SingleSpa). Conoce cómo compartir información entre Microfrontends y sabe manejar el estado, shadowDOM y mensajería entre microfrontends.	Link	Link	Link	Link	
Administración de estados	Entiende los principios Redux (única fuente de la verdad, inmutabilidad, cambios con funciones puras) y su flujo de información (acciones, disparadores, reductores, efectos y selectores). Conoce otros patrones para el manejo de estados.	Link	Link	Link		
Lenguaje	Es capaz de manipular estructuras de datos del tipo Object (Set, Date, Map, WeakMap, WeakSet). Entiende el problema de mutabilidad de la función Map.set()	Link	Link	Link		

Principios	Conoce, aplica y enseña al equipo de trabajo los conceptos y principios de Código limpio y los principios de diseño de software (SOLID, KISS, DRY, STUPID, YAGNI, HOLLYWOOD, ACID). Comprende la relación cohesión/acoplamiento .	Link	Link			
Protocolos	Se conecta a recursos externos mediante diferentes protocolos de comunicación (Websockets, GraphQL, WebRTC, gRPC).	Link	Link	Link		
Patrones de diseño	Conoce y sabe cuándo aplicar patrones de diseño Creacionales (Factory Method, Singleton), Estructurales (Fachada, Adapter) y de Comportamiento (Observer, Pub/Sub, Lazy Loading, Optimistic, Strategy).	Link	Link	Link	Link	
Paradigmas de programación	Entiende el paradigma reactivo en frameworks (reacción, control del flujo asíncrono de datos y detección de cambios). Comprende el funcionamiento de las zonas, signals, observables mediante RxJS, ChangeDetectionStrategy y data binding en Angular; Virtual DOM, setState y hooks en React; ref, reactive, proxies y data binding en Vue.	Link	Link	Link	Link	
Paradigmas de programación	Comprende y utiliza los paradigmas declarativo (map, forEach, filter, find, reduce, some, every) por encima del imperativo , dentro del contexto de Programación Funcional . Reconoce los paradigmas Smart-Dumb, Stateless-Stateful, Higher Order Components y Functional Components.	Link	Link			
Cloud	Comprende aspectos y servicios fundamentales de Cloud Computing como locations, computing, storage, databases, networking, security.	Link	Link			
Análisis de código estático	Utiliza y configura herramientas para el análisis de código estático como SonarQube, y linting de js/ts/html/css como ESLint, htmlhint o stylelint.	Link	Link			
Seguridad	Comprende conceptualmente los principios básicos de seguridad (OWASP 10) y los enseña al equipo durante las fases de diseño e implementación. Protege rutas en sus desarrollos. Entiende CORS y CSP	Link	Link	Link	Link	
Arquitectura	Conoce arquitecturas de renderización diferentes a SPA y comprende las diferencias entre hidratación y resumabilidad en SSR. Tiene la capacidad de programar aplicaciones que se renderizan tanto del lado del cliente, como del lado del servidor. Conoce las desventajas de SPA frente a SEO.	Link	Link	Link		

Experiencia	Tiene un enfoque consciente hacia los principios de experiencia de usuario (UX). Es capaz de crear controles intuitivos y fáciles de utilizar; Sus interfaces comunican constante y rápidamente sobre las acciones del usuario. Utiliza un lenguaje de usuario acorde al negocio, en orden lógico y con correcta ortografía .	Link	Link			

Promedio	Status
2.3	L1
3.5	L2
3.8	L3
5	Next

1	Deficiencia
2	Explora
3	Experimenta
4	Implementa
5	Domina

No lo ha oído, o lo ha oído pero no lo ha usado. No lo puede hacer.
Lo ha oído o ha estudiado del tema en cuestion. Lo puede hacer pero con supervisión y apoyo.
Realiza laboratorios que le enseñan como hacerlo. Puede hacerlo con algo de supervisión.
Es capaz de hacerlo sin supervisión. Lo ha implementado en algun proyecto.
Es capaz de enseñarlo.